

Architecture Decision Record

ADR-003

Idempotência via Idempotency-Key

Status: Aceite

Data	2026-05-08
Decisor	Equipa de Arquitectura
Projecto	Payment Service — Moçambique
Versão	1.0

Contexto

Em redes instáveis (contexto Moçambique), um cliente pode reenviar o mesmo pedido de pagamento por timeout ou erro de rede antes de receber a resposta do servidor.

Sem idempotência, cada reenvio criaria um novo pagamento — resultando em débitos duplicados na conta do utilizador. Num sistema financeiro, isto é um defeito crítico e inaceitável.

❑ *Requisito crítico: um utilizador que reenvie o mesmo pedido de pagamento nunca deve ser cobrado duas vezes — independentemente das condições de rede ou do número de tentativas.*

Decisão

Implementar idempotência ao nível da aplicação com um campo `idempotency_key` obrigatório no corpo do `POST /payments`, com unicidade garantida por `UNIQUE constraint` na base de dados.

```
POST /api/v1/payments
{
  "idempotencyKey": "550e8400-e29b-41d4-a716-446655440000",
  "amount": 150.00,
  "currency": "MZN",
  "phoneNumber": "+258840000000"
}
```

Comportamento HTTP

O código de resposta HTTP permite ao cliente distinguir se o pagamento foi criado agora ou recuperado de um reenvio anterior:

Cenário	HTTP	Comportamento
Primeiro pedido (chave nova)	201 Created	Pagamento criado e gravado
Reenvio (mesma chave, mesmo body)	200 OK	Pagamento original devolvido — sem duplicado
Reenvio (mesma chave, body diferente)	200 OK	Pagamento original devolvido — body ignorado

Razões

- **Segurança ao nível da base de dados** — o UNIQUE constraint em idempotency_key evita race conditions mesmo com múltiplos pods a processar pedidos em simultâneo. A aplicação verifica primeiro (findByIdempotencyKey), mas a constraint é a última linha de defesa.
- **Sem estado no servidor** — não é necessária cache distribuída (Redis) ou tabela separada de "pedidos processados"; a própria tabela payments serve de registo.
- **Comportamento previsível** — retornar o pagamento existente com 200 é mais correcto do que 201 ou erro, pois o estado é consistente independentemente de quantas vezes o cliente reenviar.
- **Rastreabilidade** — o idempotencyKey gerado pelo cliente (normalmente um UUID v4) permite correlacionar logs do cliente com logs do servidor.

Alternativas Consideradas

Alternativa	Razão de Rejeição
Header Idempotency-Key (estilo Stripe)	Requer middleware extra para extrair e indexar o header; mais complexo sem benefício adicional para este MVP
Redis para deduplicação	Adiciona dependência de infra; TTL pode expirar e permitir duplicados após longa espera
Verificação só na aplicação (sem constraint DB)	Race condition possível com múltiplos pods; não é seguro

Consequências

Responsabilidade do cliente

- O cliente é responsável por gerar um idempotencyKey único (recomendado: UUID v4) por operação de pagamento
- O mesmo idempotencyKey deve ser reutilizado em reenvios do mesmo pedido — não gerar uma chave nova por tentativa

Comportamento documentado

- Reutilizar o mesmo idempotencyKey com dados diferentes (amount, currency) retorna o pagamento original sem erro — comportamento documentado no Swagger
- O campo é indexado implicitamente pelo UNIQUE constraint, tornando as buscas por idempotencyKey eficientes

□ *Recomendação: gerar o idempotencyKey no momento de iniciar a operação de pagamento (antes da primeira tentativa), e persistir localmente para reutilizar em reenvios.*

Notas de Implementação

Lógica de idempotência no PaymentService:

```
// PaymentService.java
public PaymentResponse createPayment(PaymentRequest request) {
    // 1. Verificar se já existe (lookup por idempotency_key)
    return paymentRepository
        .findByIdempotencyKey(request.idempotencyKey())
        .map(existing -> {
            log.info("Idempotent request: {}", request.idempotencyKey());
            return PaymentResponse.from(existing); // 200 OK
        })
        .orElseGet(() -> {
            Payment payment = Payment.create(request);
            paymentRepository.save(payment); // UNIQUE constraint aqui
            eventPublisher.publish(payment);
            return PaymentResponse.from(payment); // 201 Created
        });
}
```

Referências

- ADR-001 — PostgreSQL como base de dados principal (UNIQUE constraint)
- ADR-002 — RabbitMQ para processamento assíncrono